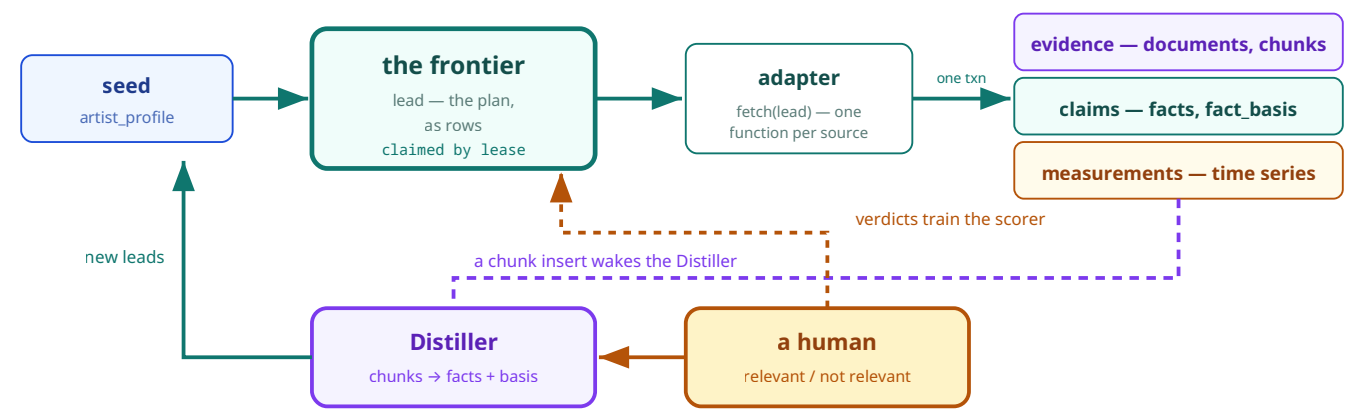


The Forager

The first agent. It researches our own artists, and builds the context every other agent will read.

Generated by docs/superpowers/specs/forager_vision.py · 2026-08-07 · source of record docs/superpowers/specs/2026-08-07-forager-agent-design.md · submission deadline 2026-08-18 17:00 EDT

An agentic OS for a record label is a fleet that is always running and always accumulating. SCOPE-RESET §1 made the artist the spine because relationships, the audience model and lessons are what compound between releases. All three have to come from somewhere. This is the agent they come from — and it is the reason the spine is not empty.



THE POINT
The cycle closes. Fetching produces leads, so the graph expands on its own; the Distiller's facts reveal what is missing, so gaps become leads; and a human's verdict changes what gets fetched next. Nothing in this picture is an orchestrator — every arrow is a row being written.

Why this agent and not the Scout

1. Scout is gated; the Forager is not	PLATFORM-SPEC §5 lists Scout first, and Scout needs SCOPE-RESET open decision 4 — counterparty acquisition — which is unresolved and marked the most urgent of the open decisions. The Forager reads sources we are entitled to read, so it can start today.
2. It makes the spine non-empty	SCOPE-RESET §1 fixed the artist as the root because relationships, audience model and lessons compound between releases. All three have to come from somewhere. This is where they come from.
3. It is the same machine the Scout needs	A lead that resolves to a person with a contact method is a counterparty candidate. Building this first means the Scout is a lead kind, not a second system.
4. It proves the memory claim a week early	The frontier is the agent's plan, stored as rows. Kill it mid-crawl and restart: it resumes. That is PLATFORM-SPEC §8's closing demo beat, available on day 3.

Who reads what it writes

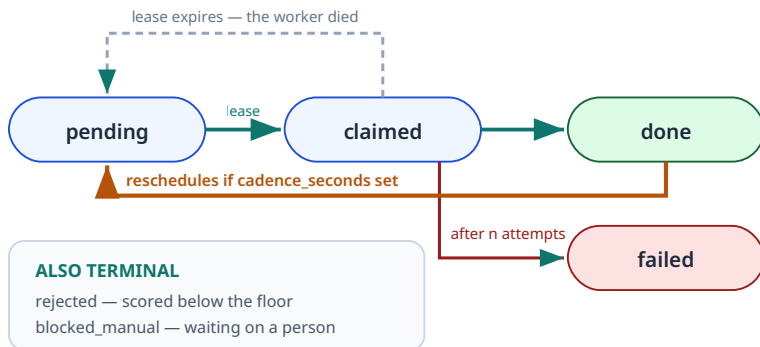
AGENT	READS	WHAT THAT BUYS
Scout	entity leads that resolved to a person with a reachable contact	counterparty acquisition stops being a blocking decision
Researcher	the enrichment pattern itself — same frontier, different lead kinds	may turn out to be the same agent
Drafter	facts, quotable chunks, and the artist's own voice	outreach that cites something real
Analyst	metric time series and the audience hypotheses the Distiller wrote	hypotheses that can be killed by evidence
RemixKit	artist identity, catalogue, brand voice	assets that match who the artist actually is

The product claim this agent has to earn. A label's advantage is not that it knows how to market one song — it is that the fiftieth release starts from everything the first forty-nine taught it. That only happens if something writes it down, keeps track of where each piece came from, and notices when it stops being true. A person with a spreadsheet does the first, rarely the second, and never the third. This is the agent that does all three, continuously, for every artist on the roster at once.

The frontier

The agent's plan is a table. Everything else follows from that one decision.

A **lead** is a typed candidate with a parent, a reason and a score. Adapters turn leads into evidence *and into more leads*. The graph expands on its own; scoring and budget bound it. A crawler can only find what someone already pointed it at — this finds the fan community nobody knew existed.



ONE COLUMN, TWO LIFECYCLES

NULL — forage once. Fetch, distil, the lead dies.
Set — measure forever. Stream counts, view counts, follower counts. Never dies.

WHY THAT MATTERS

Discovery and routine metric polling are not two subsystems that have to agree with each other. Same table, same claim, same worker.

Five properties that come free

The plan is a table	Not a context window, not a queue in another system. A recursive CTE up parent_lead_id answers <i>why were we ever looking there</i> — provenance of attention, which an agent that re-plans every run cannot answer at all.
One nullable column absorbs the scheduled crawler	cadence_seconds NULL forages once and the lead dies. Set, and it reschedules itself forever. Stream counts and view counts become the same table, claim query and worker as discovery — not a second subsystem that has to agree with the first.
Dedup and leasing are constraints, not conventions	UNIQUE (tenant_id, artist_id, target_hash) settles two Foragers discovering one handle in the same instant: one insert wins, the other retries and merges. And work is leased rather than locked, so a Forager that dies mid-fetch releases it by expiry, with no supervisor involved. PLATFORM-SPEC §3a and §3c, reused rather than re-derived.
Human work is inside the loop, not beside it	Pillar 10 §5a: no compliant API returns who used a TikTok sound. Not a dead end — a lead with mode='manual', in the same queue, scored the same way, dispatched to a person. Their verdicts train the same scorer.

Lead kinds

KIND	WHAT IT IS	MODE
profile	resolve or refresh a platform profile	auto
catalogue	releases and tracks from a metadata source	auto
metric	recurring measurement — streams, views, followers, position	auto · recurring
engagement	comments, mentions, tagged media on accounts we own	auto
mention_search	public search for the artist on any platform, including absent ones	auto
fan_artifact	a specific fan post, cover, edit or playlist	auto
document	a web page to fetch and read	auto
entity	a person or handle worth resolving — the Scout seam	auto
gap_query	a model-proposed query for a dimension we are thin on	auto
— any of the above	where no compliant API exists, the same lead goes to a human	manual

Claiming work — PLATFORM-SPEC §3a, with thread → lead, verified on the real cluster

```
UPDATE lead SET owner_agent = $agent, lease_expires_at = now() + INTERVAL '5 minutes'
WHERE id IN (SELECT id FROM lead
  WHERE tenant_id = $1 AND state = 'pending' AND next_action_at <= now()
  AND (owner_agent IS NULL OR lease_expires_at < now())
  ORDER BY score DESC LIMIT $batch FOR UPDATE SKIP LOCKED)
RETURNING *; -- partial index on state='pending'; the frontier is mostly done rows
```

The store

Three shapes, because there are three kinds of truth.

Evidence — has quotes, no truth value

TABLE	COLUMNS (ABRIDGED)	NOTE
artist_document	platform, url, content_hash, raw_key, published_at, fetched_at, http_status, gone_at	one row per thing fetched
artist_chunk	document_id, ordinal, text, embedding VECTOR(1024), token_count	the RAG surface

Claims — has truth value and provenance, no prose

TABLE	COLUMNS (ABRIDGED)	NOTE
artist_fact	dimension, value_text, provenance, status, confidence, embedding VECTOR(1024), supersedes_id	measured inferred asserted
fact_basis	fact_id, basis_kind, basis_id	chunk fact profile metric

Measurements — append-only time series

TABLE	COLUMNS (ABRIDGED)	NOTE
artist_metric	platform, entity_kind, entity_id, metric, value, observed_at	Tuesday does not supersede Monday

Configuration and judgment — what a human asserted

TABLE	COLUMNS (ABRIDGED)	NOTE
artist_profile	platform, mode, handle, credential_ref, enabled, confirmed_by	mode: owned unowned absent
artist_identifier	kind, value, valid_from, valid_until	aliases survive a rebrand
verdict	subject_kind, subject_id, relevant BOOL, judged_by	the boolean button
contradiction	claim_a_id, claim_b_id, state, resolved_by	asserted vs measured is never auto-resolved

Why not one table

A paragraph is not a claim	It is a thing someone wrote. Giving chunks a truth value they do not have is how a RAG system starts confidently reciting a 2019 blog post as current fact.
A claim cannot be quoted	Facts are distilled. If they are the only layer, every answer is a paraphrase with no way back to the sentence it came from.
Time series are neither	Monday's stream count is not superseded by Tuesday's — both are permanently true. Facts supersede; measurements accumulate. Forcing them into one shape loses one behaviour or the other.
The split is what makes change tractable	Because evidence never goes stale and only claims do, a changed fact is a status transition instead of a delete cascade. See the next page.

Vector indexes — PLATFORM-SPEC §6's amendment

```
CREATE VECTOR INDEX artist_chunk_search
  ON artist_chunk (tenant_id, artist_id, embedding vector_cosine_ops);

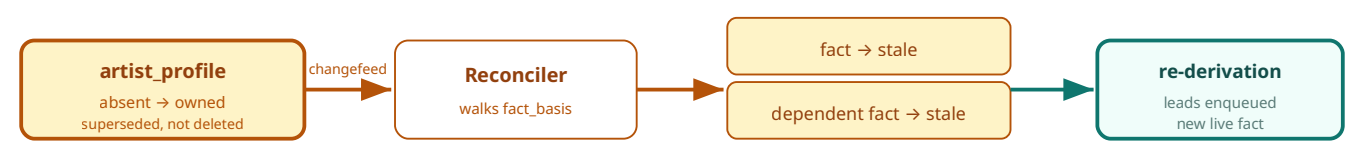
CREATE VECTOR INDEX artist_fact_search
  ON artist_fact (tenant_id, artist_id, embedding vector_cosine_ops);
```

CockroachDB accelerates a vector-index filter only on prefix columns under equality or IN. Artist scoping is always equality and is the dominant filter. platform is deliberately **not** a prefix column: adding it would break every query that does not specify a platform, and post-filtering inside one artist's chunks is cheap.

When the world moves

They had no TikTok. Now they do. Nothing is purged — staleness is a state, and it cascades.

A purge destroys exactly the audit trail a label needs when a campaign goes sideways. "We believed X on 2026-06-12 because of source S, and that ended on 2026-08-07" has to stay answerable. So facts carry a status and the change propagates through a dependency graph, rather than anything being deleted.



UNTOUCHED BY ALL OF THE ABOVE
artist_document · artist_chunk — a 2026-06 article saying "they're not on TikTok" is still a perfectly accurate quote of that article, forever. Evidence never goes stale. Only the claims standing on it do. Not one row is deleted.

AND WHERE THE SYSTEM REFUSES TO DECIDE
A TikTok search surfaces an account that looks like it is the artist — measured evidence contradicting an asserted absent. It raises a contradiction rather than flipping the config, because "the artist made a TikTok and didn't tell the label" is the finding.

A fact's status

STATUS	MEANS
live	current head of its chain — what retrieval reads
superseded	a newer version of this same claim exists
stale	basis changed or TTL expired — may still be true, no longer trusted, queued for re-derivation
retracted	a human said it was wrong

Two causes of staleness, one handler: the basis changed, or time passed — dimension_policy gives each dimension a TTL (BPM unbounded, follower count days, an audience hypothesis weeks and *should* be revised). Precedence: asserted and measured both outrank inferred, and an inferred fact may never supersede either — the most it can do is mark one stale and ask for a human.

The walkthrough

WHEN	WHAT HAPPENS	WHY IT IS BUILT THAT WAY
Day 0	A human asserts artist_profile(tiktok, absent).	Absence is a positive assertion with an author and a timestamp — not a missing row, which would mean we never checked.
Day 0	The Forager emits mention_search leads on TikTok anyway.	An absent platform is still swept for fan activity. This is the point.
Day 0	The Distiller writes an inferred fact: measurable TikTok fan activity, no artist presence — an unserved audience.	fact_basis = {the profile row, the fan-artifact chunks}.
Day 40	The artist makes a TikTok. Someone sets the profile to owned. The Reconciler walks fact_basis and marks the unserved-audience fact stale.	New profile version appended; the old one superseded, not deleted. The walk is a changefeed-driven transitive closure over a dependency graph.
Day 40	New leads seed — engagement, metric, profile refresh — and re-derivation writes a new live fact superseding the stale one.	The existing mention_search leads keep running, now more useful because fan posts can be compared against owned posts. Documents and chunks untouched; not one row deleted.

Every other kind of change

SITUATION	HANDLING
New account	Profile transition, cascade through fact_basis, new leads seeded
Rebrand	artist_identifier keeps the old name as an alias with an end date — press written before the rebrand still uses it, and the drift gate must accept it
New release	A catalogue lead seeds per-track metric leads and reopens the dimensions that depend on catalogue
Account lost or banned	owned → unowned, the same cascade in reverse
Human corrects a fact	retracted, plus a new asserted fact. Asserted always beats inferred
Source 404s on refetch	Document marked gone_at; chunks retained — we still hold the text. Confidence decays because we can no longer re-verify. Not invalidated

Why it scales

Five axes it grows along, and the three ceilings it actually has.

The scaling argument is **not** about rows per second. At one label with a small roster this database is never stretched, and any claim otherwise would be false. The argument is that every axis of growth this business has — more artists, more platforms, more throughput, more history, more purposes — is absorbed by **data or by another process**, and none of them by new code in the core.

AXIS	ABSORBED BY	HOW
Across the roster	Rows, not code	An artist with nine platforms and an artist with two run identical code, because mode and enabled are data on artist_profile. Adding artist #50 is an INSERT. Budget is per-artist, so one artist's crawl cannot starve the roster.
Across platforms	One function	A new platform is one adapter implementing fetch(lead, credentials) → documents, metrics, leads. The frontier, the store, scoring, budget, invalidation and retrieval are untouched. This is PLATFORM-SPEC §7's <i>a channel is data, not code</i> , applied to sources.
Across workers	No orchestrator to become the bottleneck	Workers are stateless; all state is rows. Throughput scales by starting more processes. FOR UPDATE SKIP LOCKED means N workers never collide and none of them needs to know the others exist.
Across time	The cost curve bends down, not up	This is the counter-intuitive one. Most crawlers get more expensive as they accumulate. Here target_hash makes re-crawls skip known targets, content_hash means an unchanged page is never re-embedded, dimension_policy means only volatile facts are rechecked, and verdicts make the scorer more precise. More history means less work per unit of value.
Across purposes	The fleet shrinks instead of growing	Scout and Researcher are both <i>expand a frontier, fetch, distil, write facts with provenance</i> . If this works, they are lead kinds rather than agents, and eight agents become six.

The one that is counter-intuitive

Most crawlers get more expensive as they accumulate. This one gets cheaper per unit of value. target_hash means a re-crawl skips every known target. content_hash means an unchanged page is never re-embedded. dimension_policy means only volatile facts are ever rechecked — BPM is never asked again. And every human verdict makes the scorer more precise, so the frontier spends its budget on better candidates than it did last month. More history means less work, not more.

The ceilings — what actually breaks first

CEILING	SEVERITY	REALITY
Vector index write rate	HIGH	The real ceiling, and it is unmeasured. PLATFORM-SPEC §10 risk 4: large batch inserts degrade the index and IMPORT INTO is unsupported on tables carrying one. One artist crawl can produce thousands of chunks in a burst. Mitigation is chunk_staging — no vector index — drained by a paced mover driven by the same lead queue. Measure before trusting; delete the staging table if direct insert holds up.
Embedding cost	MEDIUM	Scales linearly with genuinely new text. Bounded by a per-document chunk cap and by skipping near-duplicate chunks by hash, but a wide crawl on a well-covered artist is still the largest variable cost in the system.
Serializable retries under contention	MEDIUM	Serializable is the reason to choose this database and also its cost. SKIP LOCKED keeps claim contention low, but retry loops are not optional and the application has to handle them.

Cost floor: **\$0.00/month idle**. CockroachDB Basic scales to zero and a Lambda has no idle floor, so a roster nobody is working costs nothing — measured, not estimated, in infra/MEMORY-WORKLOAD.md and platform/README.md.

What the database is doing

Four jobs in one store, and what each of them buys that a second system would not.

JOB	HERE	WHAT THAT MEANS
Memory	Two vector surfaces plus the edges between them	artist_chunk.embedding (evidence) and artist_fact.embedding (claims), with fact_basis joining them — all written in one transaction. <i>Find claims like X and show me the exact source sentences</i> is one query against one store. Split across Postgres and a vector service, that is two round trips with a staleness window between them.
State	The plan is queryable	fact_basis answers where a claim came from. A recursive CTE up parent_lead_id answers why we were ever looking there. Provenance of attention is a capability, not decoration — and it is only available because the frontier is relational.
Coordination	A claim that becomes a test	Two Foragers discover the same handle from different parents in the same instant. Under Read Committed: a duplicate row or a lost score update. Under serializable plus the unique index: one lead survives with merged provenance, the loser retries. PLATFORM-SPEC §1's claim becomes a test that fails on Postgres defaults.
Event bus	The database as a build system	The Reconciler is a changefeed-driven transitive closure over fact_basis — a row change propagating invalidation through a dependency graph. Flip one artist_profile row and watch staleness fan out while re-derivation leads appear, with zero deletes.

Provenance of attention — the query an agent with a context-window plan cannot answer

```
WITH RECURSIVE trail AS (  
  SELECT id, parent_lead_id, target, reason, 0 AS hop  
    FROM lead WHERE id = $1  
  UNION ALL  
  SELECT l.id, l.parent_lead_id, l.target, l.reason, t.hop + 1  
    FROM lead l JOIN trail t ON l.id = t.parent_lead_id  
  SELECT hop, target, reason FROM trail ORDER BY hop DESC;
```

fact_basis answers *where did this claim come from*. This answers *why were we ever looking there* — back to the seed, with the reason printed at every hop.

What the Forager gives back to the platform spec

Closes PLATFORM-SPEC §10 risk 2	platform/README.md records that the RU cost of a filtered vector scan cannot be measured because <i>a probe with no rows measures nothing</i> . The Forager is the row generator — the first thing in the repository that makes that measurement possible.
Makes the Managed MCP Server demonstrable	Natural-language queries over a two-table schema demo poorly. <i>What do we know about this artist's audience in Berlin, and where did we learn it?</i> over a real dossier with citations is a different thing entirely.
Moves the closing demo beat a week earlier	The planned finale is killing the fleet mid-campaign on day 12. The Forager gives the identical beat on day 3 — kill it mid-crawl, restart, the frontier resumes — off the critical path.

Where it strains — stated rather than hidden

TENSION	STATE	RESPONSE
Changefeeds are an optimisation, not a dependency	MITIGATED	The Forager is the fleet's heaviest changefeed producer, pushing on the unverified RU draw. But the claim query already polls next_action_at, so the Forager runs correctly with changefeeds switched off entirely.
Ingest-only v1 would write vector indexes it never reads	MITIGATED	The first slice must include one retrieval query with EXPLAIN confirming prefix spans, or the headline claim goes unexercised for a week. An hour of work, not a milestone.
AS OF SYSTEM TIME is an operator tool, not a memory model	SCOPED	SELECT * FROM lead AS OF SYSTEM TIME '-4h' answers <i>what did the frontier look like before the drift</i> for free. The GC TTL on a Basic cluster bounds how far back it reaches, so it does not replace supersedes_id and is not presented as if it does.

The honest verdict on the database is the one PLATFORM-SPEC already gave, and it does not change here: at this volume Postgres would serve the workload. What CockroachDB buys is one store instead of four, correctness defaults that cannot be forgotten, and a changefeed that cannot diverge from the data it reports. That is a real engineering reason. It is not a scale reason, and presenting it as one would be the dishonest version of this document.

The submission

What it covers, what it demonstrates, what has actually been checked.

Requirement coverage

REQUIREMENT	BEFORE	WITH THE FORAGER
Distributed vector indexing	R1/R2 over counterparties — gated on open decision 4	Also R-artist over chunks and facts, with citations. Unblocked today.
Cloud Managed MCP Server	Queries over a two-table schema	Queries over a real dossier that can cite its sources.
Memory integral to agents	Threads resume after restart — day 12	Frontier resumes after restart — day 3. Same claim, off the critical path.
ccloud CLI	Not yet earned — the cluster was made in the console	Unchanged. Use it for something real or drop the claim.

Five things a judge can watch happen

BEAT	WHAT IT PROVES
Kill it mid-crawl and restart	Every lead resumes from its row by lease expiry. The fleet is stateless; the database is the runtime.
Flip one profile from absent to owned	Staleness fans out through fact_basis, re-derivation leads appear in the frontier, and nothing is deleted. The audit trail survives the change that invalidated it.
Ask why we were ever looking there	A recursive CTE walks parent_lead_id back to the seed and prints the reason at every hop. No agent with a context-window plan can answer this.
Ask the dossier a question in English	The Managed MCP Server over artist_fact and artist_chunk, answering with citations and with each claim's provenance class rendered distinguishably.
Show an audience nobody is serving	An artist with measurable fan activity on a platform where they have no account. It falls out of the schema rather than being a special report.

Verified against the real cluster, and what is not

CLAIM	STATUS	SOURCE OR ACTION
Vector indexes are enabled on Basic, and prefix filtering resolves to prefix spans	VERIFIED	platform/README.md items 1–2, against respect-the-funk-31317 (v26.2.5). Closed the flagged day-1 go/no-go, and EXPLAIN confirms the shape the retrieval design depends on.
Partial unique indexes and FOR UPDATE SKIP LOCKED both work	VERIFIED	platform/README.md items 3–4 — the two mechanisms the frontier is built out of, dedup and the lease.
RU cost of a filtered vector scan, and changefeed RU draw	UNVERIFIED	The first needs real row volume, which the Forager is what produces; the second needs a webhook sink to exist. Fallback to polling is already the claim mechanism.
Vector index degradation under burst writes	UNVERIFIED	Documented as a risk, not measured. chunk_staging exists to mitigate an unmeasured risk and should be deleted if measurement clears it.
Instagram needs no App Review for owned accounts	VERIFIED	research/01-platform-apis.md §1 — App Review and Business Verification gate Advanced Access only. Corrected an earlier assumption in this design that deferred Instagram.

Risk register

RISK	SEVERITY	MITIGATION
Twelve days, and the first slice is a migration plus five adapters	HIGH	Bounded deliberately — ingest only, judged in SQL : the migration, the frontier with lease and budget, five adapters, chunk-and-embed, one Distiller pass, one retrieval query with EXPLAIN. No console, no retrieval API, no verdict buttons. Everything else, including the Reconciler cascade and the human surface, is the second slice.
Topical drift — an artist name matches unrelated text	HIGH	The failure mode that actually kills a forager, and it is not volume. Identifier anchoring rejects anything past depth 2 reaching no canonical ID through no owned edge, and a cheap relevance check runs before the expensive fetch-and-embed.
Vector index write rate under burst	HIGH	The sharpest tension between this design and this database. chunk_staging plus a paced mover, and a measurement before the mover is trusted or deleted.
A model writes a fact that no source supports	MEDIUM	Every fact requires at least one fact_basis row, so a fact with no basis is rejected at write. Hallucination becomes a constraint violation, not a silent claim.
Two things we must not overclaim	MEDIUM	Counterparty acquisition (SCOPE-RESET open decision 4) has a mechanism in the manual lead mode, so it can be decided later — but it is not resolved and is not presented as resolved. And any curve that looks like an improvement metric gets labelled with its N: the house rule that made screen_clips.py abstain applies to our own demo.